

Chap 1

1) How structured design works?

Step 1 - Divide the problem into smaller problems

Step 2 - Analyse each subproblem and obtain a solution to solve the sub problem.

Step 3 - The solutions of all the subproblems are then combined to solve the overall problem.

2) How Object-Oriented Design Works?

Step 1 - Identify objects for the problem

Step 2 - Define data and operations for each object

Step 3 - Establish interactions between objects

3) Object - An object consists of its unique state (data), and behavior (methods)

Object data - State of an object is represented by data fields with their current values

Object operations - Object operations is invoked on an object to perform a specific action.

4) Basic principles of OO Design

a) Encapsulation - Bundling data (variables) and methods into a single unit.
- Data Hiding, i.e. private __, protected __

b) Inheritance - The ability to create new classes from existing classes.
- Promotes Code Reusability and establishes a "is-a" relationship.

c) Polymorphism - The ability to use the same expression to denote different operations.
- Enhances flexibility

5) Class VS Object

Class

- A template or blueprint that defines the data fields and methods an object will have.

Object

- A concrete instance of class

are interchangeable

VS: An Object is an instance of a class. The terms 'object' and 'instance' are interchangeable

Chap 2

1) Method Signature

Method signature is the combination of the method name and the parameter list.

Exp: `max(int num1, int num2)`

2) Formal parameters

The variables defined in the method header are known as formal parameters.

3) Actual parameters // argument

When a method is invoked, pass a value to the parameter. This value is called as argument.

4) Modifiers

Modifiers are the reserved words that specify the properties of the data, methods and classes and how they can be used.

Exp: `public, private, static, abstract`

5) Public access VS Private access

- Private makes methods and data fields ~~accessi~~ accessible only from within its own class.

- Public makes classes, methods and data fields accessible from any class.

6) Static variable

- Are shared by all the instances of the class.

7) Method Overloading

- Two or more methods with the same name but different parameter lists.

Exp: `public static int max(int num1) // public static double max(double num1)`

8) Benefits of Methods

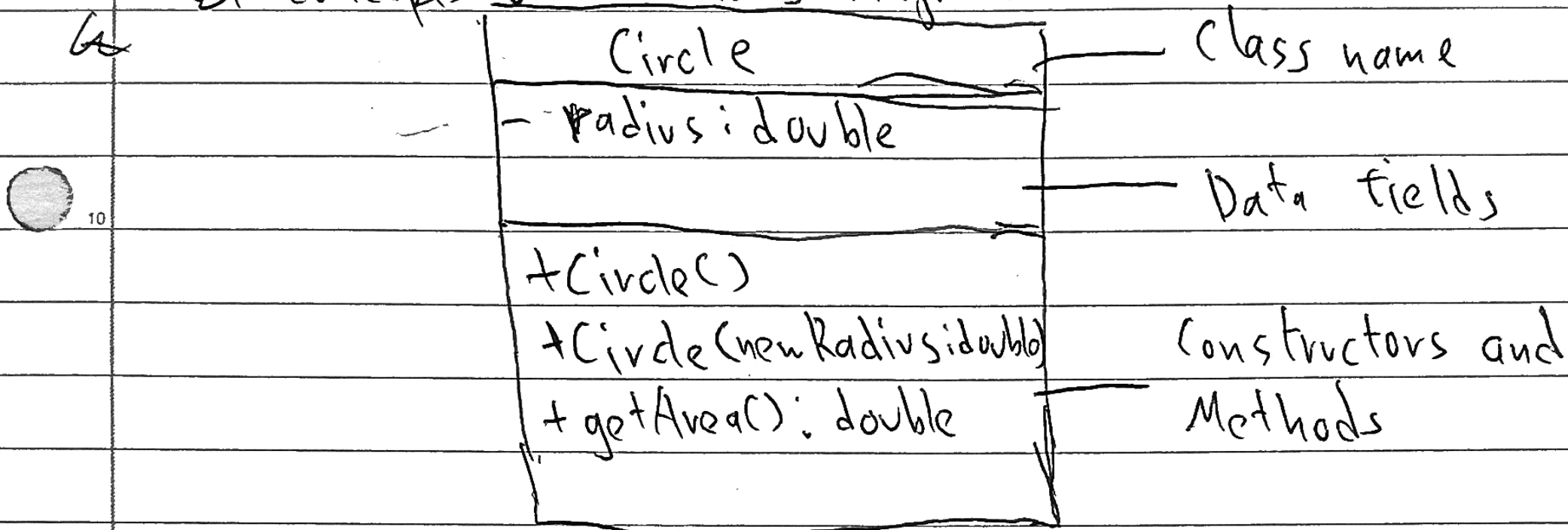
- Reduce complexity
- Ease of maintenance

a) Random Method

```
(char)('A' + Math.random() * ('F' - 'A' + 1));  
return a random character between A and F
```

- 1) Procedural Paradigm (PP) Vs Object-oriented paradigm (OOP)
- Data & methods are loosely coupled.
 - Software design focuses on ~~the~~ designing methods.
 - Couples data and methods together into objects
 - Software design focuses on objects and operations on objects

2) ~~OOP Concepts~~ UML Class Diagram



3)

F

~~slides~~

slides

Chap 4 (b)

Ref:

Date:

1) ~~4~~ Date

`java.util.Date // java.util.Date date = new java.util.Date();`

2) ~~Math~~ `java.util.Random // Math.random()`

3) Static variables VS ~~Instance~~ constant variables

— A static variable is shared among all objects of the particular class that declares the static variable.

— A constant variable is a variable whose value cannot be changed after it has been initialized

4) Modifier

— The private modifier restricts access to within a class

— The default modifier restricts access to within a package

— The public modifier enables unrestricted access.

5) Accessor / Mutator Methods

• Accessor methods

— Methods used to read private properties.

Exp: getter or get methods

• Mutator Methods

— Methods used to modify private properties.

Exp: setter or set methods

1) Class Design Guideline

• Cohesion

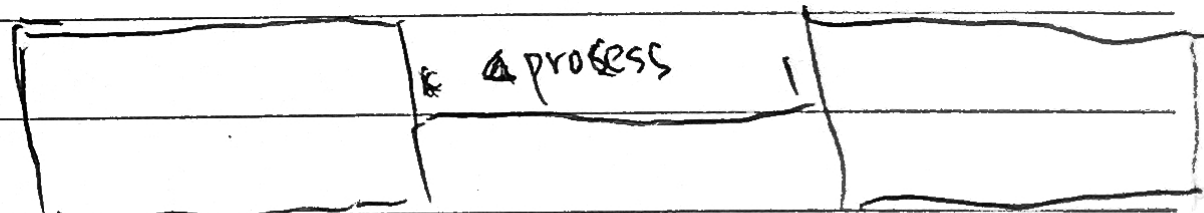
— A class should describe a single entity.

• Consistency

— Follow standard java programming style and naming conventions.

• Encapsulation

— Class should ~~use~~ use the private modifier to hide its data from direct access by clients.



2) Association

— A general relationship that describes an activity between two classes.

3) Aggregation

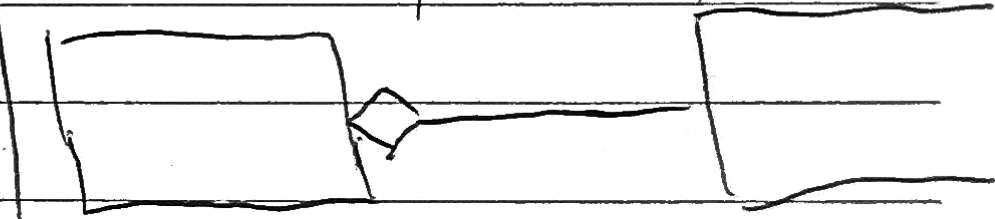
— An object can contain another object

— Aggregation models has-a relationship

— An object may be owned by several

~~other~~ other aggregating objects.

Expi: Many to Many / one to Many

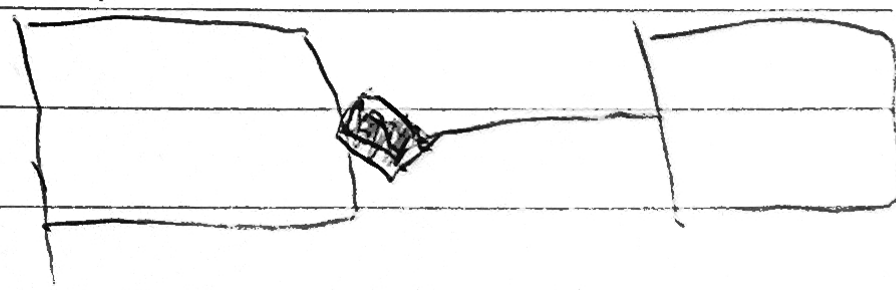


— typically involves ~~one~~ one to many relationship

4) Composition

— If an object is exclusively owned by an aggregating object, the relationship between them is referred to as composition.

Expi: one to one



— typically involves one to one relationship

1) ~~Procedural~~ Programming 4 Visibility modifiers

private | The data or methods can be accessed only within the declaring class

default -

protected -

public | The class data or method is visible to any class in any package

2) This keyword (this.)

- The this keyword is the name of a reference that refers to a calling object itself.
- to reference a class's hidden data fields

1/27

slides
(4(d))